QCon
全球软件开发大会

InfoQ 技术大会

AI Agent 执行环境的规模化难题

CubeSandbox 的解法与实践

龙进

腾讯云 CubeSandbox 核心 Maintainer

腾讯高级研发工程师



龙进

腾讯云 CubeSandbox 核心 Maintainer · 腾讯高级研发工程师

龙进，腾讯云高级工程师，拥有丰富的云计算研发经验

长期专注于沙箱与 Serverless 基础设施方向

深度参与 CubeSandbox 沙箱系统的研发建设

在沙箱、OS Kernel、弹性 GPU 等核心方向有丰富的研发及项目经验



沙箱与 Serverless
云计算基础设施长期深耕

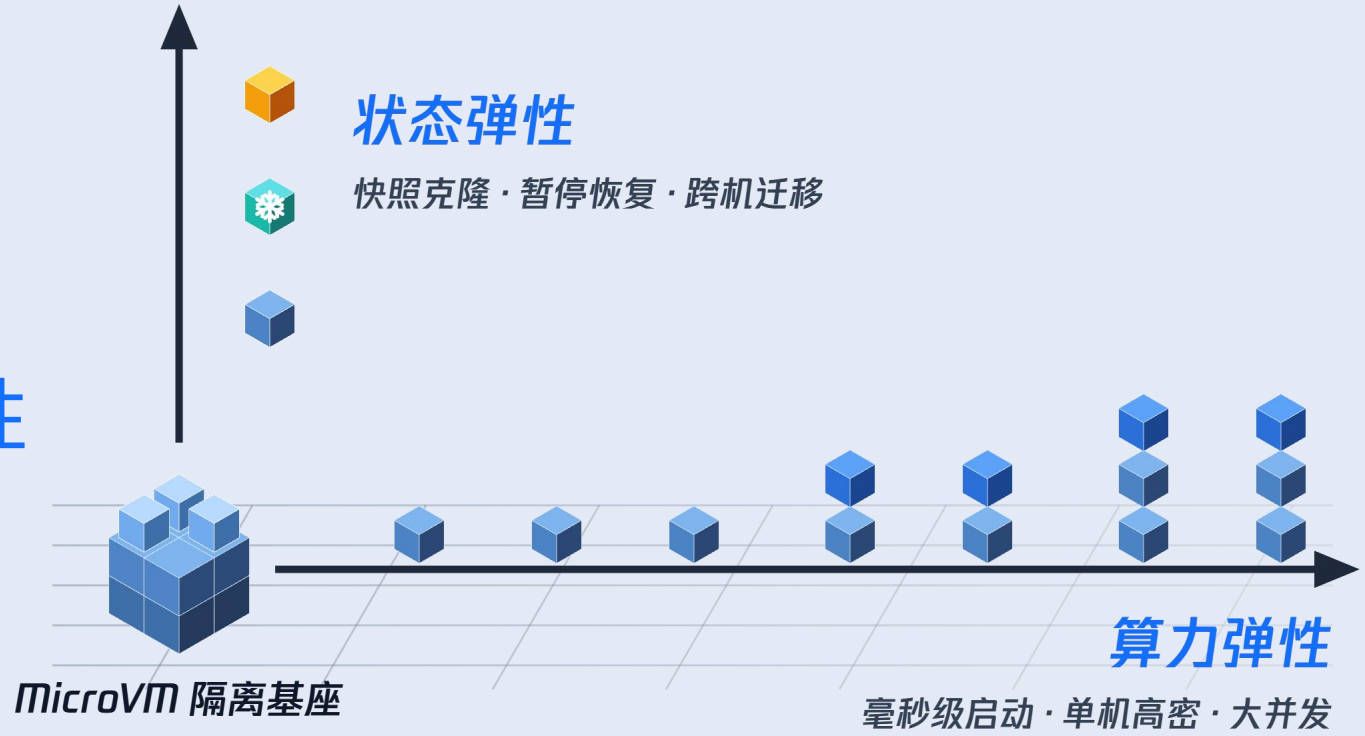
CubeSandbox 开源
沙箱系统核心架构与建设

OS Kernel 与弹性 GPU
内核与异构算力研发实践

弹性沙箱计算

以 MicroVM 为安全基座，通过统一快照原语打通算力弹性与状态弹性

以沙箱为基本计算单元
兼具算力弹性与状态弹性



QCon
全球软件开发大会

InfoQ 技术大会

目录

01 问题与挑战：Agent 工作负载的三重演进

02 选型与架构：MicroVM 强隔离与快照优先

03 算力弹性：百毫秒拉起 · 高密部署 · 百万级并发

04 状态弹性：快照克隆 · 暂停唤醒 · 跨机迁移

05 生产落地：大规模验证与集群部署

06 总结与展望：核心结论与开源演进

01

问题与挑战

从单次工具调用到长周期、有状态、大规模并行

Agent 工作负载演进：时长维度

从秒级单次函数调用，走向数小时乃至跨天的长周期会话



Agent 工作负载演进：状态维度

从无状态工具箱，走向沉淀文件、进程与上下文的有状态延续



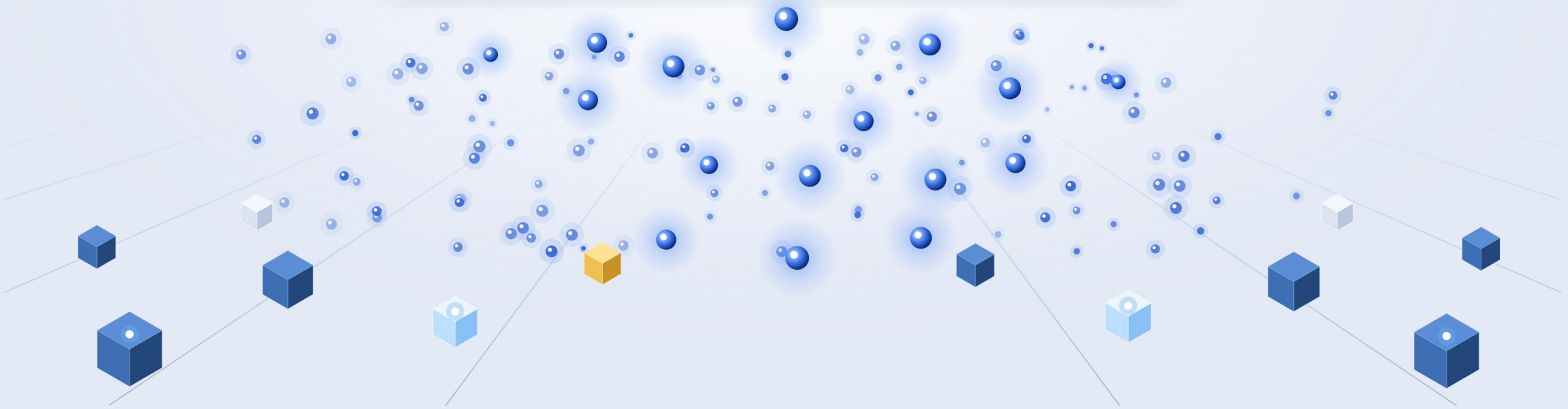
Agent 工作负载演进：规模维度

从零星单任务调用，走向瞬时成千上万沙箱的大规模并行

Agent 工作负载演进 | 规模演进

单任务 ----> 大规模并行

RL Rollout / 大规模评测 · 瞬时成千上万个隔离沙箱并发拉起



三类场景的需求画像

时长、状态、规模三种演进叠加，分化出三类典型负载与核心指标

场景类型	生命周期	并发形态	状态需求	最关键的指标
工具执行 代码 · 浏览器 · 数据分析	秒级 — 分钟级	高频、平稳调用	基本无状态	启动延迟 P99 单机密度
训练与评测 RL Rollout · SWE-bench	分钟级	瞬时万级洪峰	固定起点 · 可分叉复现	创建吞吐 批量调度耗时
长周期 Agent 个人助手 · 企业平台	数小时 — 跨天	常驻 · 大量空闲	内存 · 文件 · 技能沉淀	暂停唤醒 跨节点迁移 空闲成本

为什么现有基础设施难以兼顾

容器平台、传统云主机与 Serverless 函数，各自存在难以跨越的架构短板

现有方案	独立内核隔离	百毫秒级就绪	高密低开销	有状态长会话	核心架构短板
容器平台 Docker / K8s Pod	✗ 共享内核	✗ 秒级就绪	✓ 开销极低	✓ 支持常驻	共享宿主内核缺安全边界；容器冷起与业务启动仍需数秒
传统云主机 KVM / 传统 VM	✓ 独立内核	✗ 数十秒至分钟级	✗ 内存开销大	✓ 支持常驻	完整 OS 引导与初始化耗时数十秒至分钟级，单实例开销大
Serverless 函数 FaaS 平台	✓ 强隔离	✓ 预热百毫秒级	✓ 高密弹性	✗ 空闲即销毁	面向无状态短任务设计，空闲回收后会话与环境状态丢失

安全前提

MicroVM 强隔离

独立内核 · 硬件级虚拟化边界

\+

核心能力一

算力弹性

百毫秒拉起 · 高密部署 · 万级并发

\+

核心能力二

状态弹性

暂停唤醒 · 跨机迁移 · 快照克隆

02

选型与架构

以 MicroVM 强隔离为基座，构建快照优先的沙箱架构

主流沙箱方案对比：同样是 MicroVM，核心差异在哪

安全隔离方案已经收敛，真正的分水岭在于创建链路、单机密度与状态管理

对比维度 核心能力拆解	容器平台（基线） Docker / K8s Pod	K8s + Firecracker 通用编排 + 轻量虚机自建	E2B 云端托管沙箱服务	CubeSandbox 专为 Agent 设计的沙箱底座
安全隔离边界	✕ 共享宿主内核 易受内核提权与逃逸威胁	✓ MicroVM 独立内核 KVM 硬件虚拟化隔离	✓ MicroVM 独立内核 KVM 硬件虚拟化隔离	✓ MicroVM 独立内核 叠加 eBPF 与 L7 流量管控
创建与启动链路	− 秒级就绪（数秒） 拉取镜像 + CNI + 业务冷起	− 秒级以上 多跳异步调度 + 完整 OS 引导	✓ 百毫秒级 托管控制面 + 模板快照恢复	✓ 48 ms 极速就绪 极简调度链路 + 快照池化恢复
单机部署密度	− 百级实例 受 Pod 配额限制，无跨实例内存共享	− 百级实例 受 Pod 开销限制，无跨 VM 内存共享	● 单机指标未公开 SaaS 托管黑盒，按实例时长计费	✓ 实测 2000+ 实例 内存 CoW / PageCache / 磁盘全复用
状态快照与克隆	✕ 仅静态镜像打包 不含运行态内存，不支持状态回滚	✕ 编排层无原生支持 需自建底层快照与存储分发体系	✓ 支持快照、暂停与克隆 支持运行态快照、按需暂停恢复与克隆	✓ 原生百毫秒快照 零拷贝 1:N 分支克隆与原地秒级回滚
交付与部署形态	● 开源自建 / 云容器 复用现有 K8s 运维体系	− 深度自研改造 需绑定裸金属并改造 K8s 组件	● 公有云 SaaS 为主 提供标准 E2B SDK 接口	✓ Apache 2.0 全面开源 兼容 E2B · 支持 K8s / 裸机 / PVM 云主机

CubeSandbox 的三大核心技术支柱

以轻量定制 MicroVM 为安全基座，通过全生命周期快照与专用调度同时兑现两种弹性

01

安全与部署基座

轻量定制虚拟化层

基于 Cloud Hypervisor 深度裁剪

- **极致精简冗余设备模型**
  单实例 VMM 内存开销压至 < 5 MB
- **PVM 赋能普通云服务器部署**
  无需裸金属支持，部署更灵活、算力资源更易按需补充
- **完整保留高阶硬件扩展**
  支持网卡/磁盘热插拔与 GPU 直通（自研 PVDMA 映射）

核心架构收益

独立内核强隔离 · < 5 MB 极低开销 · 裸机与云主机全兼容

02

打通算力与状态弹性

全生命周期快照引擎

一套快照原语贯通创建、分叉与跨机迁移

- **告别冷引导，快照极速拉起**
  从模板快照直接恢复（含业务进程），按需加载内存
- **运行态增量快照与零拷贝克隆**
  XFS Reflink + Soft-Dirty，百毫秒捕获并支持 1:N 分叉
- **支持跨机按需暂停恢复**
  空闲自动暂停落盘释放算力，请求到达时跨机按需恢复

核心架构收益

48 ms 极速就绪 · 1:N 零拷贝克隆 · 跨机按需暂停恢复

03

支撑百万并发与超高密度

专用调度与全栈复用

绕开通用编排多跳链路与资源隔离墙

- **CubeMaster 极简调度链路**
  Local Cache + Redis/MySQL 多级缓存，支持百万并发扩展
- **节点资源预池化极速拼装**
  网卡、磁盘与快照提前池化就绪，单机毫秒级闭环拼装
- **跨沙箱全栈资源复用**
  内存 mmap 写时复制 + PageCache 唯一 + 磁盘去重

核心架构收益

百万并发水平扩展 · 单机实测 2000+ 实例高密承载

冷启动只发生一次

直接从内存与磁盘快照恢复，省略传统 Guest OS 启动，支撑五项核心能力

核心结论 一

一次构建
随处极速恢复

直接从内存与磁盘快照恢复

省略传统 Guest OS 启动过程

让完整的冷启动只发生一次



统一快照原语

01 极速创建 模板快照直接拉起 · 48 ms

02 并发克隆 运行态 1:N 零拷贝极速分叉

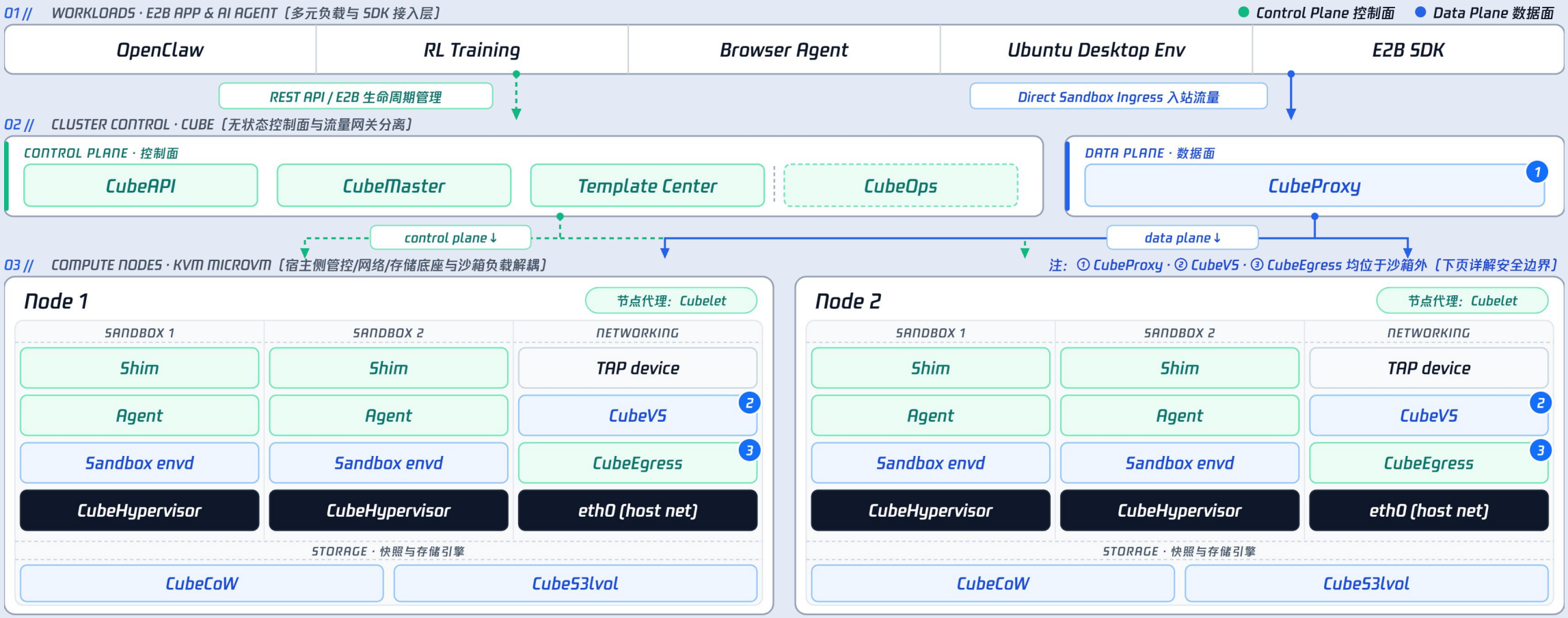
03 状态回滚 故障或误操作后秒级还原

04 按需暂停恢复 空闲暂停落盘，请求到达恢复

05 跨机无损迁移 经共享存储跨节点暂停恢复

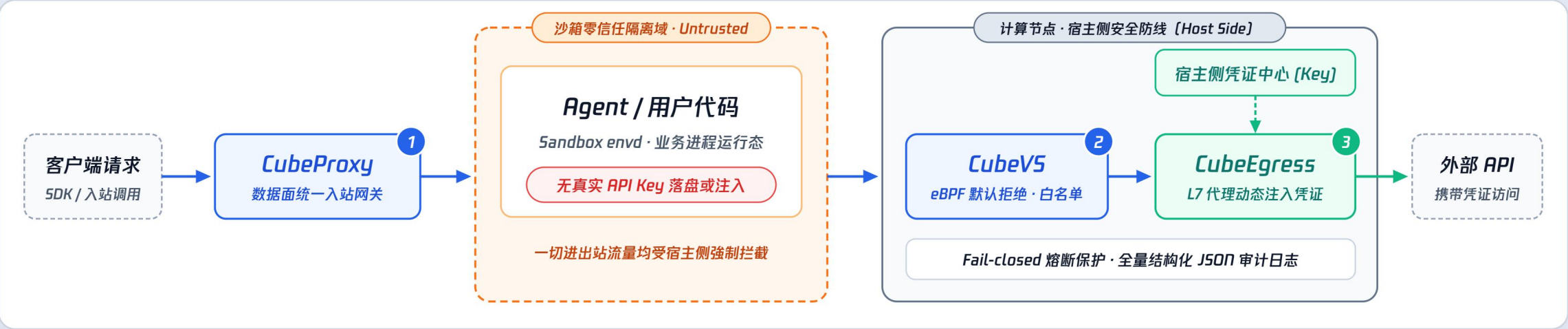
CubeSandbox 整体系统架构

控制面与数据面分离，宿主侧网络/存储底座与沙箱负载解耦



网络与凭证安全：进出站策略与敏感密钥全部收归宿主侧

将沙箱内部视为零信任域，API 密钥不落盘、进出流量强制管控与审计



1 CubeProxy · 数据面入口

入站流量统一收敛

所有沙箱入站请求必经统一数据面网关

集中鉴权与按需路由，阻断外部直连探测

2 CubeVS · 内核态 eBPF

零信任出网访问控制

出站连接默认拒绝，仅放行白名单域名与 IP

具备 Fail-closed 故障熔断与全量流量审计

3 CubeEgress · L7 出口代理

敏感凭证动态注入

出网阶段从宿主侧凭证中心动态注入真实 Key

沙箱内零密钥落盘，客户密钥轮换免重建模板

内核隔离与安全职责边界：硬件级防逃逸与爆炸半径兜底

以独立内核收敛底层攻击面，与上层平台授权体系形成纵深防御闭环

底层执行环境虚拟化隔离

共享内核容器 vs 独立内核 MicroVM

传统容器方案（Docker / K8s Pod）

共享内核 · 逃逸风险

容器实例 1 (Agent) | 容器实例 2 (Agent) | 容器实例 N (Agent)

共享宿主机内核 (Shared Host OS Kernel) · 系统调用直达宿主

痛点：庞大的宿主内核系统调用面暴露，单点内核提权漏洞即危及整机与邻租户

CubeSandbox 硬件级虚拟化隔离

独立内核 · 物理级隔离

沙箱 1 · 独立 Guest 内核 | 沙箱 2 · 独立 Guest 内核 | 沙箱 N · 独立 Guest 内核

CubeHypervisor (KVM) | CubeHypervisor (KVM) | CubeHypervisor (KVM)

CPU 硬件虚拟化边界 (Intel VT-x / AMD-V) · 宿主机内核彻底解耦

收益：攻击面收敛至极简 Hypervisor 边界，恶意系统调用仅限当前沙箱

全栈安全职责边界划分

上层管「业务授权」，沙箱管「环境兜底」

上层 Agent 平台与工具网关

负责：能做什么

· 负责业务身份认证、工具调用审批与操作权限裁决

· 决定 Agent 能否调用 kubectl、能否读写特定生产数据库与 API

CubeSandbox 沙箱执行底座

负责：出错不扩散

· 沙箱不是业务授权系统，不耦合上层业务逻辑裁决

· 提供独立内核、零信任网络与无密钥环境三道物理级安全围栏

核心安全承诺：即使上层授权失误或遭遇提示词注入劫持
破坏与爆炸半径也严格锁死在当前单沙箱内部

03

算力弹性

百毫秒极速拉起 · 单机高密部署 · 万级并发调度

冷启动路径

把该做的事，延迟到真正需要的时候再做

传统启动



数秒至数十秒（OS 引导 + 业务进程初始化），按同一比例要延伸到屏幕之外很远

Cube



1. **取出池化好的网卡和磁盘**
   资源提前建好，只做拼装
2. **从快照恢复**
   跳过系统引导
3. **内存按需加载**
   没碰过的页，不建映射

放大后的 48 ms，不按比例

测试环境：腾讯云 BMI5 裸金属 · 沙箱 2 vCPU / 2 GiB · cube-bench · 单并发 avg 48 ms / P95 57 ms

CubeMaster 极简调度与多级缓存

绕开通用编排多跳异步链路，以多级缓存架构支撑百万并发水平扩展

K8s + Firecracker 通用编排链路

异步多跳 · Watch 与 etcd 瓶颈

Client 发起请求 → apiserver 鉴权/准入 → etcd 写对象落盘 → scheduler Watch 选点 → kubelet Watch 感知 → MicroVM CNI/冷启动

痛点：每一跳均需写 etcd 并等待下一环异步 Watch；大规模突发创建下易触发 etcd 写瓶颈与 Watch 风暴

CubeSandbox 极简调度链路

内存级同步决策 · 一跳直达节点

Client / SDK 创建请求 → CubeAPI 接入网关 → CubeMaster Local Cache 选点 → Cubelet 本地池化拉起 → MicroVM 48 ms 就绪

Redis + MySQL 多副本状态协同

1s 周期上报节点视图

优势：无 etcd、无 Watch 风暴；基于沙箱专属策略〔模板本地性、创建并发、MVM 配额〕内存级同步返回

百万级

100k+ 分钟级完成调度

并发调度与控制面水平扩展能力

01 Local Cache 极速决策

CubeMaster 内存维护实时节点视图

过滤 → 打分 → 选定全内存同步完成

02 Redis + MySQL 多级协同

多调度模块间通过 Redis + MySQL 协同

彻底摆脱 etcd 写入吞吐天花板限制

03 无状态多副本水平扩展

控制面组件无状态设计，按需水平扩容

结合单机资源池化，支撑百万并发调度

高并发下的长尾延迟治理

单次启动 48 ms 只是起点，通过三层系统治理在 50 并发下守住 137 ms P99

单机高并发创建闭环：将资源初始化前置入池，关键路径内聚并行

高并发请求
50 并发瞬时到达
→ ① 节点资源池直接取用
网卡池 · Cgroup 池 · 磁盘池
→ ② 内聚与并行装配
Cubelet 内聚网络（少 2 次 RPC）
‖ 沙箱快照恢复与宿主配置并行
→ 稳定就绪
P99 137 ms

底层抗抖动保障：③ 全栈锁粒度优化 + Cube 与宿主机系统服务共存针对性优化

137 ms
50 并发 P99 延迟
avg 67 ms（较单并发仅 +19 ms）

180.9 个/s
20 并发峰值创建吞吐
单沙箱均摊耗时仅 5.5 ms

01 资源分配层 单机资源预池化

网卡池化 · Cgroup 池化 · 磁盘池化

沙箱所需的虚拟网卡、Cgroup 与磁盘全部提前池化，创建时直接从池中取出拼装

将耗时且易冲突的资源初始化移出关键路径，规避高并发下的实时分配争抢

02 编排执行层 关键路径内聚与并行下发

内聚减 2 次 RPC · 并行下发

网络运行时直接合入 Cubelet 进程，每个沙箱减少两次跨进程 RPC，配合 eBPF 极速下发

沙箱启动与宿主侧配置并行下发、不串行等待，压缩高并发下的排队等待时间

03 宿主环境层 全栈锁与系统服务共存优化

全栈锁优化 · 系统服务共存优化

系统性优化并发创建路径上的锁竞争，提升高并发负载下的延迟稳定性

针对 Cube 与宿主机系统服务共存时可能造成的影响做针对性优化，消除长尾毛刺

单节点阶梯并发实测：创建吞吐与延迟表现

测试环境：腾讯云 BMI5 裸金属（96 核 / 375 GiB）· 沙箱 2 vCPU / 2 GiB · cube-bench



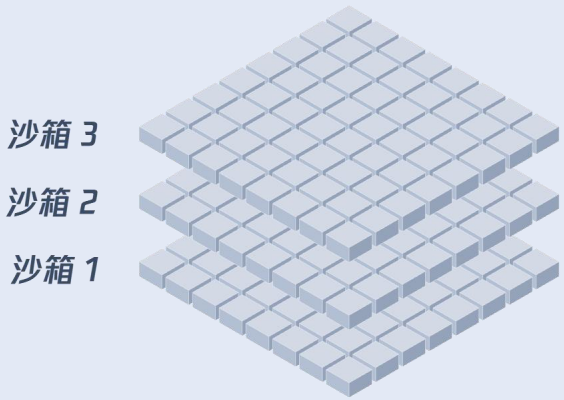
1 → 50 并发全部请求 100% 成功：平均耗时仅从 48 ms 增至 67 ms，P99 稳定在 137 ms

高密度内存模型：只读共享 + 写时复制

基于同一模板创建的沙箱，未修改的内存页与磁盘块整机只存一份

传统 VM

每个沙箱各持有一整份内存与磁盘



CubeSandbox

整机共享一份只读快照 + 各自写过的脏页



< 5 MB
单实例额外内存开销

2000+
96 核物理机实测存活沙箱

3000+
单节点软件设计上限

高密度的前提：Agent 负载天然具备间歇性错峰特征

沙箱绝大部分时间在等待大模型响应，不超卖就无法释放真实的物理机密度潜力

多沙箱并发运行态时间轴：工具执行短脉冲 vs 模型推理长等待



负载规律：同一时刻仅有少数沙箱处于活跃执行峰值，静态 1:1 锁死配额将造成大量算力与内存闲置

核心结论 二

不超卖，
高密度就无从谈起

单机 2000+ 高密承载来自两级复用的叠加：

第一级杠杆：写时复制〔CoW〕

复用同模板沙箱间「未修改的只读内存页」

消除基础镜像与运行时的静态重复占用

第二级杠杆：差异化资源超卖

复用沙箱在「等待模型期间的闲置配额」

错峰共享物理算力，大幅突破静态配额上限

高密度下的稳定性治理：CPU 与内存差异化超卖及水位保护

针对软瓶颈与硬红线实施差异化超卖策略，通过真实物理水位准入与动态回收守住节点底线

01 软瓶颈 · 仅变慢不崩溃

CPU：支持高倍率超卖

CFS 时间片排队

多沙箱峰值争抢时分时复用

仅变慢、不崩溃

不破坏进程上下文，不中断任务

策略：高倍率激进超卖

充分压榨整机多核利用率

02 硬红线 · 严防宿主 OOM

内存：审慎控制超卖比

突破上限触发 OOM

物理内存耗尽将强杀沙箱进程

状态丢失、不可接受

有状态长周期会话直接中断

策略：稳健控制超卖比

严守物理红线，配套动态回收

03 动态准入 + 主动回收闭环

兜底：真实物理水位保护

防线一：真实水位准入

逼近物理内存阈值时动态拦截调度

防线二：内存自动回收

基于内存气球自动回收空闲页

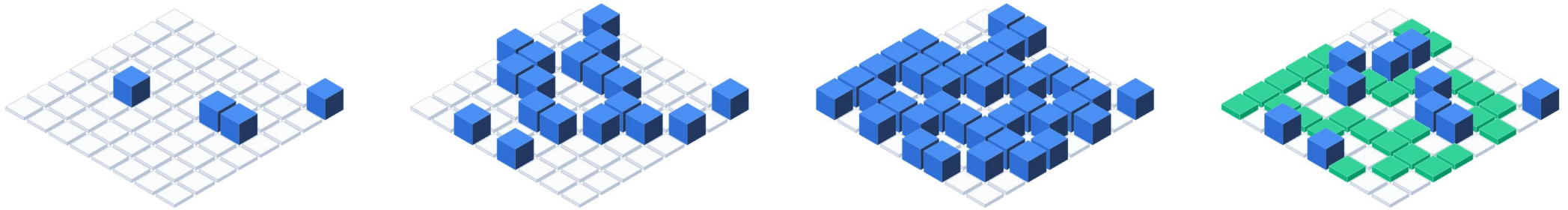
防线三：空闲 AutoPause

会话空闲落盘，彻底释放物理内存

长周期沙箱的内存治理：写时复制分裂与内存自动回收

基于内存气球（Memory Balloon）实现内存自动回收，结合 AutoPause 治理脏页膨胀

只读共享页〔整机共用一份〕　实例独占私有页〔写时复制分裂产生〕　内存气球〔Memory Balloon〕已自动回收归还宿主的内存页



① 刚从快照启动
全量共享只读页 · 极少私有页

② 运行中〔装依赖/编译〕
写入触发 CoW 分裂 · 脏页增长

③ 业务执行峰值
大量内存页分裂为实例独占页

④ 内存自动回收后
基于内存气球回收空闲页归还宿主

挑战：长周期运行脏页膨胀

为什么运行越久，独占私有页越多？

- MAP_PRIVATE 写时分裂：沙箱首次修改某页时，即从共享页分裂为实例独占页
- 业务运行持续写入：安装依赖、编译构建、写日志及临时文件均会触碰新内存页
- 传统痛点：Guest OS 内释放内存后，若无主动回收机制，宿主机侧仍计为已占用

解法：内存自动回收与多维协同

CubeSandbox 动态内存回收闭环

- 基于内存气球〔Memory Balloon〕的自动回收：主动回收沙箱空闲内存归还宿主
- 真实水位调度 + 空闲 AutoPause：按实际物理内存记账，会话空闲自动暂停落盘
- rootfs 走 virtio-pmem + DAX：绕开 Guest PageCache，从源头减少冗余私有页

04

状态弹性

毫秒级快照克隆 · 自动暂停唤醒 · 跨节点无损迁移

沙箱角色的范式演进：从一次性执行容器到持久化运行环境

越来越多的生产级 Agent 将完整工作流放入沙箱长期运行，状态弹性从可选项变为刚需

早期形态 · 工具容器

一次性执行容器（用完即毁）

· 每次调用临时新建 → 跑单条命令 → 立即销毁

· 痛点：跨轮次文件与进程丢失，依赖反复重装

演进

当前主流 · 运行环境

会话级持久化运行环境（长期常驻）

· 技能快照秒级拉起 → 多轮状态延续 → 空闲暂停保活

· 价值：文件、进程与上下文跨小时/跨天完整保留

核心结论 三

状态弹性不是锦上添花
而是长周期 Agent 的刚需

基础设施必须为会话状态的持久化、分叉与流转负责

01 开源 24k+ Stars 知识平台

WeKnora 知识智能体

演进前 docker run --rm 每次新建，跨轮状态丢失

演进后 会话级常驻沙箱 + 技能快照化，空闲暂停保活

02 复杂研发与办公工作流

某企业研发 Agent

演进前 每次会话临时拉起容器，无法备份与回退

演进后 守护进程 + Agent 整体常驻，支持随时快照回滚

03 长周期量化与数据分析

某企业金融投研 Agent

演进前 仅将单步代码片段发往外部容器执行

演进后 模型—工具主循环全量入沙箱，长周期安全闭环

支撑长周期运行：状态弹性的四大核心技术命题

围绕沙箱状态的保存、分叉、流转与重连，构建端到端有状态生命周期引擎

01 技术命题 一

运行态分支与秒级回滚

像管理 Git 分支一样管理沙箱环境

提供快照、克隆、回滚三大原语，支持 1:N 并行探索与故障重置

性能实测：从快照创建 64 ms · 状态回滚 82 ms

02 技术命题 二

毫秒级增量状态持久化

快照开销与磁盘大小解耦，恢复耗时与状态大小无关

磁盘基于 XFS Reflink 零拷贝，内存基于 Soft-Dirty 仅转储增量脏页

性能实测：增量快照 50 ms · 按需恢复 42–85 ms

03 技术命题 三

算力与状态按需解耦

空闲时零算力占用保活，恢复时支持跨节点无损调度

会话空闲触发 AutoPause 落盘释放算力；结合共享存储实现跨机按需暂停恢复

性能实测：网关拦截请求自动唤醒 P99 < 100 ms

04 技术命题 四

恢复路径外部环境无缝重连

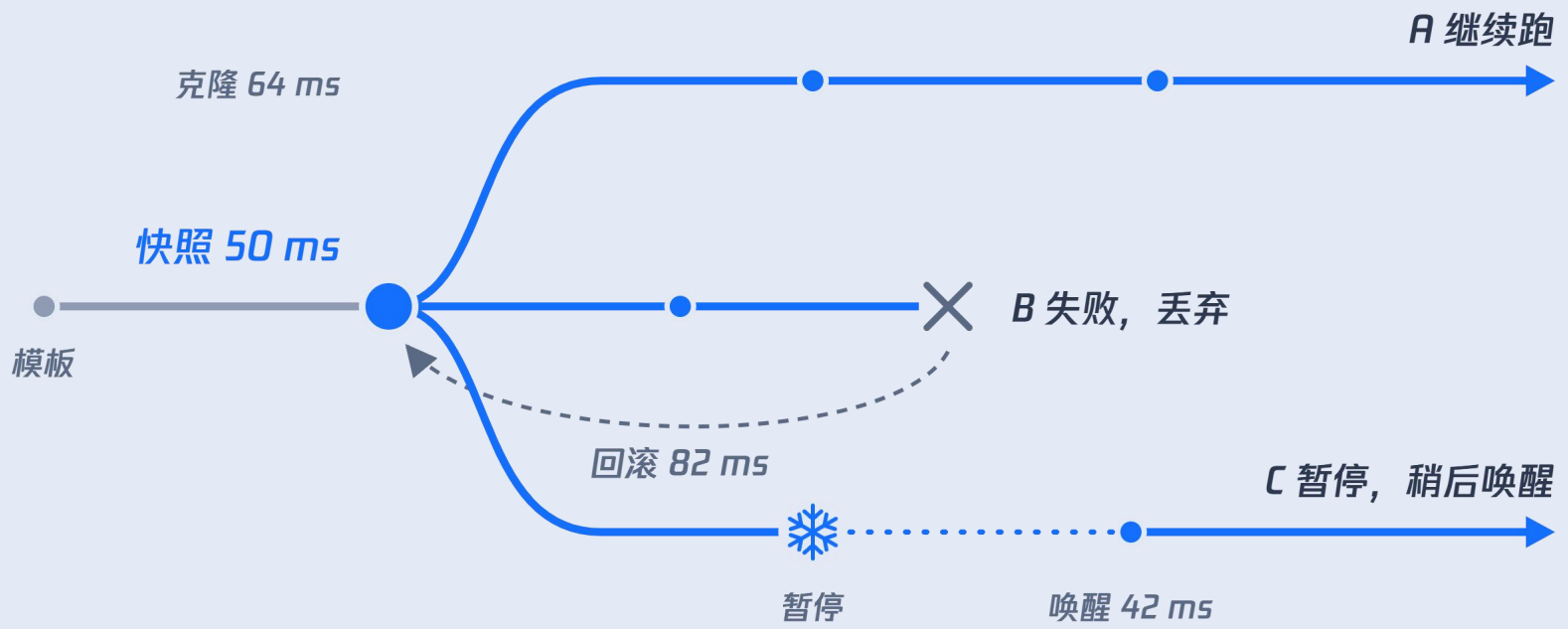
不重跑 entrypoint，在恢复路径上重建外部连接与配置

快照恢复仅还原沙箱内部状态；宿主侧绑定的外部资源需在恢复路径上重新接通

核心机制：池化网卡与网络规则重绑 · 动态身份与环境变量注入

像管理 Git 分支一样管理沙箱运行环境

基于快照、克隆、回滚三大原语，支持并行路径探索与故障秒级重置



三类核心恢复原语

从快照启动
基于快照新建或 1:N 克隆实例

按需唤醒
暂停落盘沙箱恢复至内存运行态

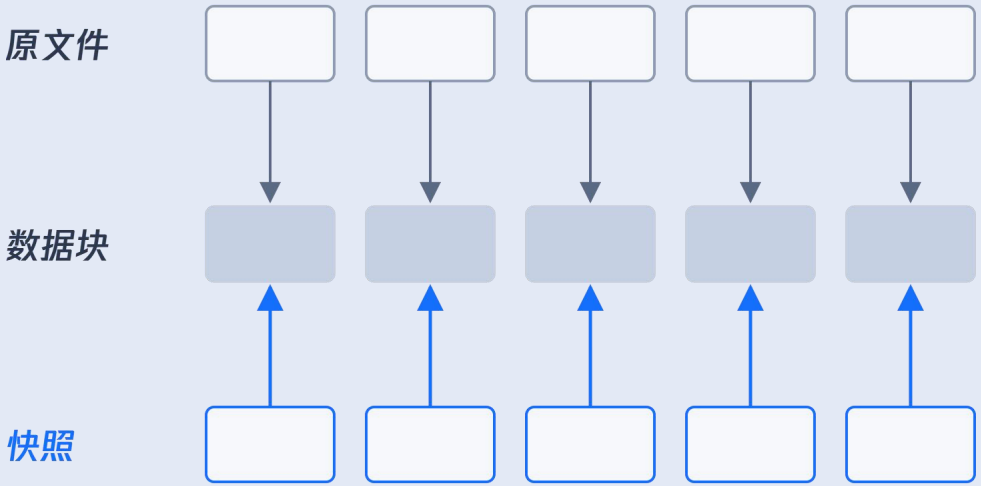
状态回滚
运行中沙箱原地还原至历史快照

测试环境：腾讯云 BMI5 裸金属 · 沙箱 2 vCPU / 2 GiB · cube-bench · 基于快照创建 64 ms · 状态回滚 82 ms

毫秒级快照的底层原理

磁盘基于 XFS Reflink 零拷贝克隆元数据，内存基于 Soft-Dirty 仅转储新增脏页

磁盘卷快照（XFS Reflink）



仅复制指向数据块的元数据目录，耗时与文件大小无关

内存页快照（Soft-Dirty 增量追踪）

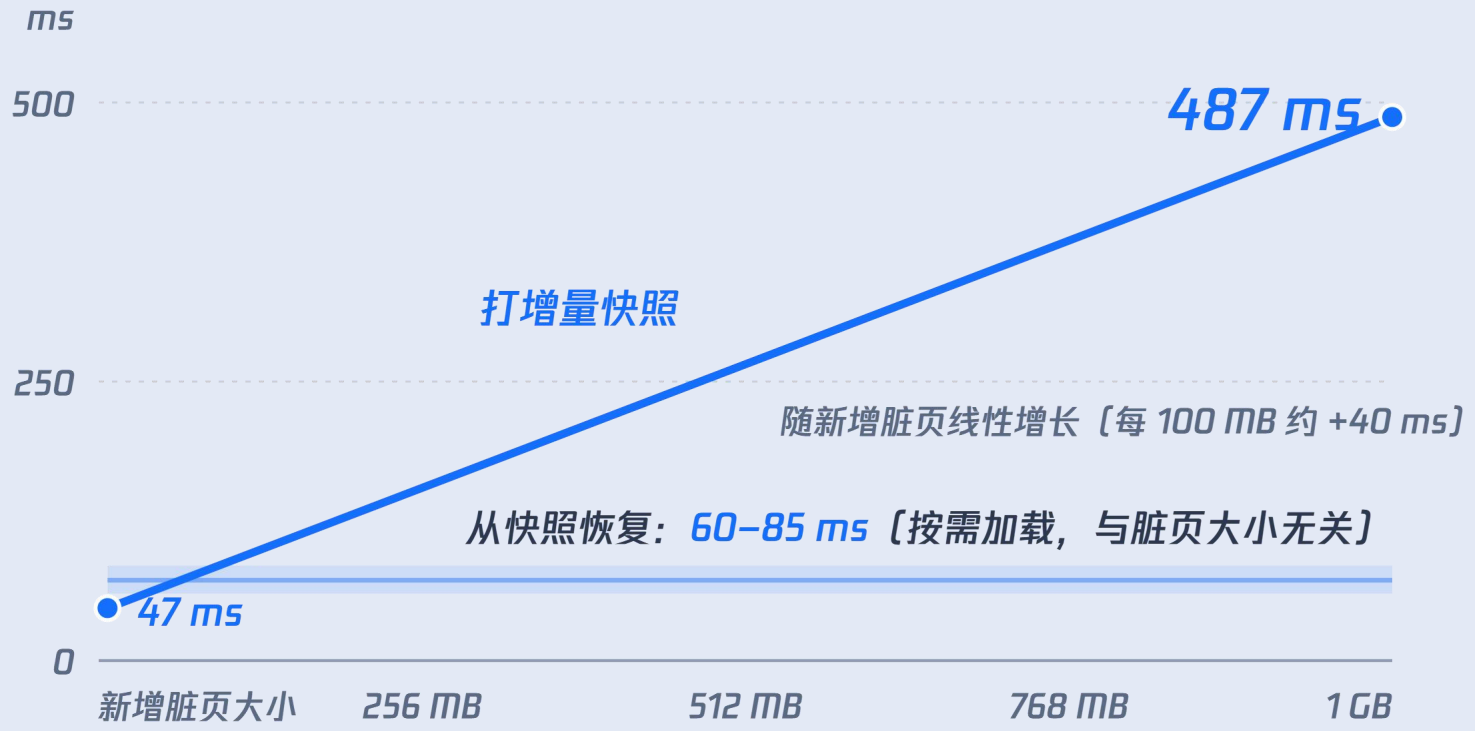


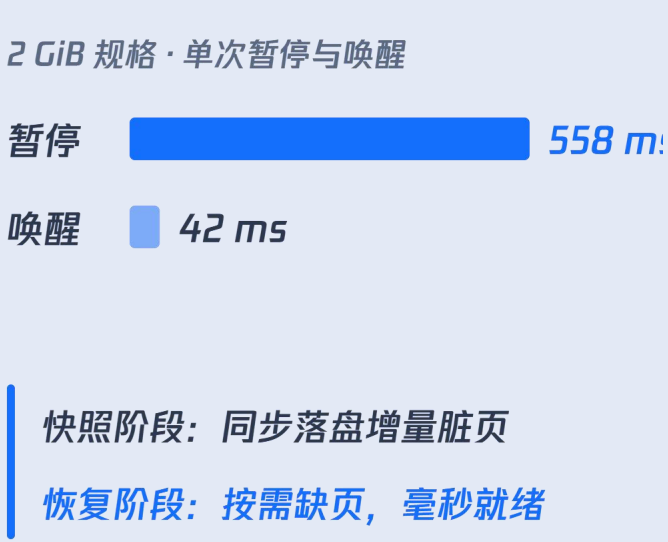
三级高可用兜底：Soft-Dirty 增量 → 实例独占页 → 全量转储，确保快照操作稳定可靠。

关键机制：XFS reflink（FICLONE）元数据克隆 · 内存 MAP_PRIVATE 写时复制 · pagemap 读取 soft-dirty 位

快照与恢复的耗时特征：增量脏页落盘 vs 按需缺页恢复

快照耗时仅取决于本轮新增脏页量；恢复采用按需缺页加载，耗时恒定在百毫秒内





测试环境：腾讯云 BMI5 裸金属 · 沙箱 2 vCPU / 2 GiB · cube-bench · 曲线两端为实测值

生命周期管理：自动暂停恢复与跨节点迁移

会话空闲时自动落盘释放算力，节点维护或故障时跨机无损恢复



空闲自动暂停（AutoPause）将闲置沙箱的资源归还宿主机，显著提升单机密度。

快照恢复后的外部资源重连

从快照恢复不会重跑 entrypoint，与外部绑定的网络和配置需动态重连



使用建议：业务进程需支持暂停恢复后的外部长连接重连。

沙箱暂停较长时间后再恢复时，原有的数据库或外部连接可能已超时断开。

05

生产落地实践

大规模生产环境验证、生态兼容与集群部署方案

核心性能指标与生产实测规模

在大规模生产环境中验证的业务收益与集群承载规模

1 亿+

元宝生产环境用户规模

95.8%

元宝 AI 编程场景核时消耗下降

87%

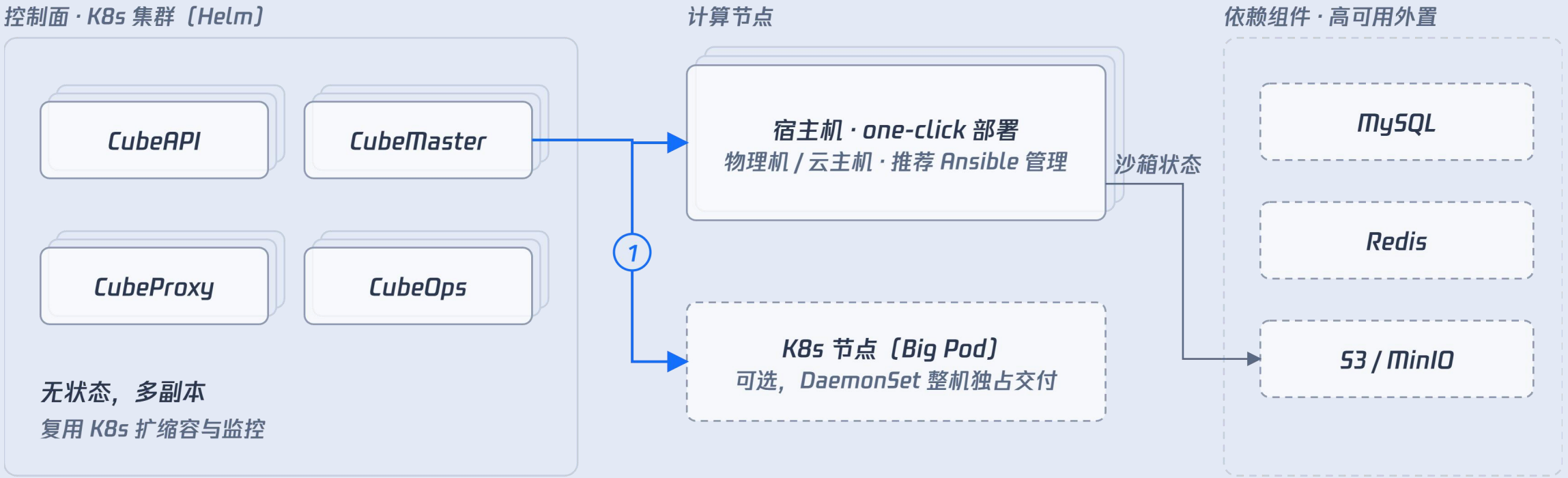
某头部大模型公司训练效率提升

100k+

沙箱分钟级完成调度

推荐部署方式

控制面 Helm 部署于 K8s，计算节点支持宿主机直装与 K8s Big Pod



1 两种计算节点注册到同一套控制面，统一调度

宿主机集群推荐结合 Ansible 批量运维

支持 x86_64 和 ARM64 • MySQL、Redis、MinIO 建议用云上产品化服务，或由部署方自己保证高可用

训练与评测：瞬时洪峰

通用编排撑不住洪峰；快照让每个评测从同一个起点出发

8,192

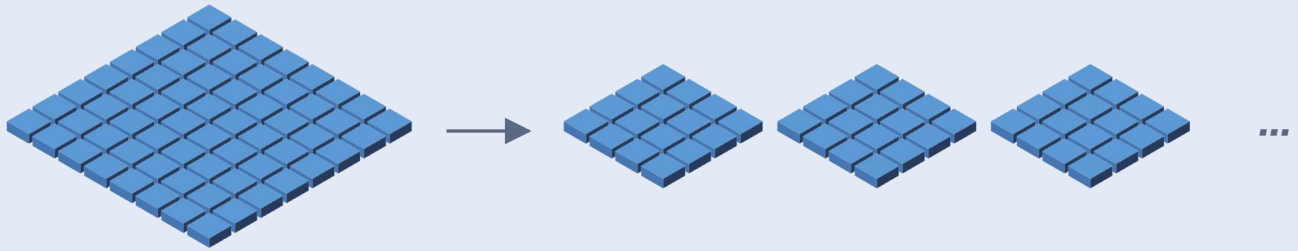
个沙箱，要在 rollout 阶段瞬时拉起

原方案：K8s 上自建

多 Controller 对账 → etcd 成为瓶颈

冷启动被拖垮，沙箱起不来

GPU 闲置，等环境



一个起点快照　　*N 个一模一样的评测环境*

固定起点做成快照 → 并行拉起 → 测试判分 → 销毁

客户环境

E2B 兼容与迁移

改两个环境变量，业务代码不用动

为什么兼容 E2B

迁移成本是选型时最大的阻力之一，
所以不另起一套协议。

三语言 SDK 能力已拉齐

Python / Go / Node

迁移案例（客户环境）

从 Daytona 迁移：启动从 10 秒以上
降到 100 毫秒以下

```
# 只改两个环境变量

- export E2B_API_URL=<原 E2B 地址>
+ export E2B_API_URL=<your-cube-endpoint>

- export E2B_API_KEY=<原 E2B 密钥>
+ export E2B_API_KEY=<your-cube-api-key>
```

06

总结与展望

核心技术结论、选型评估清单与开源演进路线

三个核心结论

以 MicroVM 与统一快照为底座，同时兑现算力弹性与状态弹性

01 冷启动只发生一次，随处极速恢复

48 ms
单次冷启动耗时

02 只读共享、错峰超卖与内存自动回收

2000+
单机实测存活沙箱

03 从一次性执行容器演进为运行环境

< 100 ms
跨节点按需唤醒 P99

选型时值得问的六个问题

01 安全隔离边界

共享宿主内核，还是独立内核？

能否防御内核提权与逃逸？

02 单机并发长尾

高并发创建下的 P99 延迟是多少？

而不只是看单个沙箱冷启动有多快

03 长稳运行密度

真实负载持续运行数小时之后，

单机还能稳定承载多少实例？

04 集群水平扩展

集群节点规模扩大之后，

并发吞吐能否随节点数线性增长？

05 状态快照与迁移

快照恢复耗时是否与状态大小无关？

暂停的沙箱能否跨节点按需唤醒？

06 外部凭证安全

Agent 访问外部 API 与服务时，

敏感密钥会不会明文进入沙箱？

开源与演进：从 v0.1 到本年度内整体 GA

13k GitHub Stars · GitHub Trending 全语言榜 Top 4 · CNCF Landscape 官方收录

v0.1 – v0.3 — 核心底座
MicroVM 极速启动
快照 · 克隆 · 回滚

v0.4 – v0.6 — 安全与云原生
L7 凭证注入 · 自动暂停
ARM64 · K8s 与 Volume

v0.7.0 – v0.7.3 — 当前最新 · 高可用
控制面全链路高可用 · 跨机恢复
内存自动回收 · 原生 MCP 支持

本年度内整体 GA — v1.0 里程碑
生产级全面就绪
异常恢复 · 调度增强 · 存储优化

01 高可用与容灾

沙箱异常自动恢复

自动检测沙箱异常，支持重启或回滚快照等策略

计算节点发生故障时，支持沙箱自动跨节点恢复

02 集群智能运维

调度与运维能力增强

支持资源感知调度、亲和/反亲和规则与优先级

支持在线资源均衡与带沙箱迁移的节点排空

03 性能与成本优化

跨机恢复与存储性能优化

加快快照传输并降低跨机恢复时延，逼近本机速度

支持 S3 增量上传、本地缓存与纯文件系统快照

社区与生产采用

已被广泛采用：腾讯 · MiniMax · Xiaomi · 联想 · 港大 HKUDS · 云知声 · 唯品会 · 花椒直播 · 芒果TV 等

开源社区与 CUBE 100 计划

分钟级跑通你的第一个沙箱，欢迎参与开源共建与生产落地交流

官方渠道与文档入口

Apache 2.0 全面开源

开源仓库 GitHub	github.com/TencentCloud/CubeSandbox
官方网站 Website	cubesandbox.com
技术文档 Docs	cubesandbox.com/docs
官方动态 X / Twitter	@CubeSandbox

CUBE 100 计划

首批 100 个落地团队 · 限席开放

已在生产环境运行 AI Agent？诚邀加入首批 100 个共创团队

提供架构共创、性能优化支持与核心研发团队一对一答疑

申请与详情 → 扫码添加右侧「微信小助手」

Star · Try · Contribute

GitHub 开源仓库

欢迎 Star 关注与提交 Issue / PR

TencentCloud/CubeSandbox

CUBE 100 计划申请入口

微信小助手

CUBE 100 计划详情与申请

用户群 · 演讲资料 · 一对一答疑

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

github.com/TencentCloud/CubeSandbox

